

From the Ground Up

Derivatives $\rightarrow$ Backprop $\rightarrow$ Neural Networks

A Complete Lecture, Built in Layers

10 Sections — From First Principles to Image Classification

Abstract.

This document presents a complete derivation of backpropagation from first principles, structured as a ten-part lecture. Beginning with the definition of the derivative and progressing through the chain rule, gradient descent, and the autograd engine, it builds intuition layer by layer until arriving at neural networks and their training. The material is designed for readers with no prior knowledge of deep learning—only basic calculus and programming are assumed. Every concept is motivated by a question, introduced with a concrete example, and formalized as code. By the end, the reader will understand why a neural network learns, and how the mathematics of the chain rule makes it all possible.

Contents

1	Derivatives	5
1.1	The Question	5
1.2	The Definition	5
1.2.1	Common Derivatives	5
1.3	The Linear Approximation	5
1.4	The Learning Rule	6
1.4.1	The Sign, Not the Magnitude	6
2	The Chain Rule	7
2.1	Compositions	7
2.1.1	Intuition: Ratios Cancel	7
2.2	Concrete Example	7
2.3	Branching	8
2.4	Why This Is the Whole Game	8
3	Gradient Descent	9
3.1	The Optimization Problem	9
3.2	The Update Rule	9
3.3	The Learning Rate η	9
3.4	Why It Works	10
4	Gradients (Many Inputs)	11
4.1	From Scalar to Vector	11
4.2	The Update Becomes Elementwise	11
4.3	The Gradient Points Uphill	11
4.4	The Big Realization	11
5	The Autograd Engine	13
5.1	The Value Class	13
5.2	Local Derivatives as Code	13
5.2.1	Addition	13
5.2.2	Multiplication	13
5.3	Why +=?	14
5.4	Derived Operations	14
5.5	Forward Pass Builds a Graph	14
5.6	Backward Pass Uses the Chain Rule	14
6	Backprop	15
6.1	The Setup	15
6.2	Topological Order	15
6.3	Seeding and the Reverse Walk	15

6.4	Why This Is Efficient	15
6.5	Backprop = Reverse-Mode Autodiff	16
6.6	The Full Picture	16
7	Neural Networks	17
7.1	A Neuron	17
7.2	Why Nonlinearity?	17
7.3	A Layer	17
7.4	The MLP	18
7.5	Parameter Count	18
7.6	Every Forward Pass Is a Composition	18
8	The Loss	21
8.1	What a Loss Does	21
8.2	Cross-Entropy for Classification	21
8.3	Why Log + Softmax?	21
8.4	Why Not MSE?	21
8.5	Numerical Stability	22
9	The Training Process	23
9.1	Step 1: Define the Model	23
9.2	Step 2: Prepare Data	23
9.3	Step 3: The Loop	23
9.4	The Three Phases	24
9.4.1	Zeroing Gradients	24
9.5	What Changes as the Model Learns	24
10	Putting It All to Work	25
10.1	The Dataset	25
10.2	Results	25
10.3	What the Model Learned	25
10.4	What We Skipped	26
	Summary	27
	Exercises	29
	References	31

Chapter 1

Derivatives

1.1 The Question

Key Idea. The entire motivation for derivatives in deep learning can be stated in one sentence: *You have a function $f(x)$. You can change x . You want to make $f(x)$ smaller. Which way should you move x ?*

This question—and its answer—is the entire foundation of neural network training. Every algorithm, every architecture, every optimization technique in deep learning exists to answer it more efficiently.

1.2 The Definition

Definition 1.1 (Derivative) *The derivative of a function f at a point x is defined as:*

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (1.1)$$

In plain English, Equation (1.1) asks: “If I nudge x by a tiny amount h , how much does f change, per unit of nudge?”

The derivative $f'(x)$ is a number that captures the **slope** of f at x . It tells you two things simultaneously: the direction (sign) in which f increases, and the rate (magnitude) at which it changes.

1.2.1 Common Derivatives

Table 1.1: Common derivative rules.

Function $f(x)$	Derivative $f'(x)$	Meaning
x^2	$2x$	At $x = 3$, slope is 6
$3x + 1$	3	Constant slope
$\sin(x)$	$\cos(x)$	Oscillating slope
e^x	e^x	Slope equals value

1.3 The Linear Approximation

The most important practical consequence of the derivative is the **linear approximation**:

$$f(x + h) \approx f(x) + f'(x) \cdot h \quad (1.2)$$

Note. The linear approximation says: “*New value $\approx$ old value + (slope) $\times$ (how far I moved).*” This is the only first-order approximation that matches f at x and has the correct slope. For **small** steps, it lets us predict how f responds to any perturbation.

1.4 The Learning Rule

Given a derivative, we can now answer the original question:

- If $f'(x) > 0$: increasing x increases f , so we should **decrease** x .
- If $f'(x) < 0$: increasing x decreases f , so we should **increase** x .

Both cases are captured by a single rule:

$$x_{\text{new}} = x - \eta \cdot f'(x) \quad (1.3)$$

where $\eta > 0$ is a small constant called the **learning rate**. Equation (1.3) is gradient descent in one dimension—and it is the seed from which all of neural network training grows.

1.4.1 The Sign, Not the Magnitude

Note. The derivative’s **sign** tells you direction. Its **magnitude** tells you steepness.

- Large $|f'(x)|$ means a steep slope—big steps are safe.
- Small $|f'(x)|$ means a flat region—small steps, or you will overshoot.

When we scale by η , we are accounting for both the direction and the terrain simultaneously.

Chapter 2

The Chain Rule

2.1 Compositions

Key Idea. Real functions are compositions. The chain rule is the tool that lets us differentiate them.

Suppose we have a chain of dependencies:

$$u = 3x + 1, \quad v = u^2, \quad f = v + 5 \quad (2.1)$$

The question is: **how does x affect f ?** The chain rule answers this directly:

$$\frac{df}{dx} = \frac{df}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{dx} \quad (2.2)$$

2.1.1 Intuition: Ratios Cancel

Note. Think of each derivative as a ratio of small changes:

$$\frac{\Delta f}{\Delta x} = \frac{\Delta f}{\Delta v} \cdot \frac{\Delta v}{\Delta u} \cdot \frac{\Delta u}{\Delta x} \quad (2.3)$$

The intermediate differentials Δv and Δu cancel like fractions. It is arithmetic bookkeeping of “how a small change ripples through each stage.”

2.2 Concrete Example

With $x = 2$:

$$u = 3 \cdot 2 + 1 = 7, \quad \frac{du}{dx} = 3 \quad (2.4)$$

$$v = 7^2 = 49, \quad \frac{dv}{du} = 2u = 14 \quad (2.5)$$

$$f = 49 + 5 = 54, \quad \frac{df}{dv} = 1 \quad (2.6)$$

Applying the chain rule:

$$\frac{df}{dx} = 1 \cdot 14 \cdot 3 = 42 \quad (2.7)$$

Note. Verification: computing directly, $f(x) = (3x + 1)^2 + 5$, so $f'(x) = 2(3x + 1) \cdot 3 = 6(3x + 1) = 6 \cdot 7 = 42$. ✓

2.3 Branching

If f depends on x through multiple paths, the contributions **sum**:

$$f = g(x) + h(x) \implies \frac{df}{dx} = \frac{dg}{dx} + \frac{dh}{dx} \quad (2.8)$$

Note. This is why every `_backward` method in our autograd engine uses `+=` instead of `=`. A value may contribute to multiple paths, and all contributions must be accumulated.

2.4 Why This Is the Whole Game

Key Idea. Every neural network is a deeply nested composition.

Loss = $L(\text{MLP}(\text{image}))$

The chain rule lets us compute $\partial \text{Loss} / \partial \theta$ for every parameter θ by multiplying local derivatives along the path from θ to the loss.

Chapter 3

Gradient Descent

3.1 The Optimization Problem

We want to minimize a loss function parameterized by θ :

$$\theta^* = \arg \min_{\theta} L(\theta) \quad (3.1)$$

We cannot solve this analytically (the function is nonlinear and has millions of parameters). We cannot try every combination (the space is infinite). But we can:

1. Compute $\frac{dL}{d\theta}$ at the current θ .
2. Take a small step in the **opposite** direction.
3. Repeat.

3.2 The Update Rule

$$\theta \leftarrow \theta - \eta \cdot \frac{dL}{d\theta} \quad (3.2)$$

- If $\frac{dL}{d\theta} > 0$: increasing θ increases $L \implies$ decrease θ .
- If $\frac{dL}{d\theta} < 0$: increasing θ decreases $L \implies$ increase θ .
- If $\frac{dL}{d\theta} = 0$: you are at a critical point (possibly a minimum).

3.3 The Learning Rate η

Table 3.1: The effect of the learning rate.

η too small	Training crawls
η too large	Overshoot, oscillate, or diverge
η just right	Smooth descent to a local minimum

Typical values range from 0.001 to 0.1. It is **the single most important hyperparameter**.

3.4 Why It Works

For small η , the linear approximation (Equation (1.2)) holds. Each step provably decreases L up to second-order terms. Gradient descent is **local search guided by a global mathematical tool**—the derivative.

Chapter 4

Gradients (Many Inputs)

4.1 From Scalar to Vector

When L depends on many parameters $\theta = (\theta_1, \theta_2, \dots, \theta_n)$, the derivative becomes a vector:

$$\nabla L = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_n} \right] \quad (4.1)$$

Each component answers: “If I nudge only θ_i , how much does L change?”

4.2 The Update Becomes Elementwise

$$\theta_i \leftarrow \theta_i - \eta \cdot \frac{\partial L}{\partial \theta_i} \quad \text{for every } i \quad (4.2)$$

This is still just “move each parameter opposite to its own derivative.” The gradient simply packages all these individual derivatives into a single object.

4.3 The Gradient Points Uphill

Key Idea. A beautiful fact from multivariable calculus: ∇L points in the direction of **steepest ascent**. Therefore $-\nabla L$ points downhill.

Gradient descent walks the steepest path down.

4.4 The Big Realization

Key Idea. For a neural network with millions of parameters, computing $\partial L / \partial \theta_i$ naively (by perturbing each θ_i separately) would require **millions** of forward passes.

Backprop computes **all** gradients in **one backward pass**. The total cost is approximately twice the forward pass—independent of the parameter count. This is the trick that makes deep learning feasible.

Chapter 5

The Autograd Engine

5.1 The Value Class

Key Idea. Every number in our computation is a **Value**. Each remembers how it was created and how to propagate gradients backward.

```
1 class Value:
2     def __init__(self, data, _children=(), _op=''):
3         self.data = data
4         self.grad = 0.0
5         self._backward = lambda: None
6         self._prev = set(_children)
7         self._op = _op
```

Listing 5.1: The Value class skeleton

Each **Value** stores: (1) its **data**, (2) its **gradient** $\partial L / \partial \text{self}$ (starts at 0), (3) how it was **created**, and (4) a **_backward** function to propagate gradient.

5.2 Local Derivatives as Code

Every operation knows its own derivative. We encode it in **_backward**.

5.2.1 Addition

```
1 def __add__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data + other.data, (self, other), '+')
4     def _backward():
5         self.grad += 1.0 * out.grad
6         other.grad += 1.0 * out.grad
7     out._backward = _backward
8     return out
```

Listing 5.2: Addition: $\partial c / \partial a = 1$, $\partial c / \partial b = 1$

5.2.2 Multiplication

```
1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4     def _backward():
5         self.grad += other.data * out.grad
```

```

6     other.grad += self.data * out.grad
7     out._backward = _backward
8     return out

```

Listing 5.3: Multiplication: $\partial c / \partial a = b$, $\partial c / \partial b = a$

5.3 Why +=?

If a value is used multiple times, gradients from all uses must **sum**. Consider:

$$f = a \cdot b + a \cdot c \implies \frac{\partial f}{\partial a} = b + c \quad (5.1)$$



The value a feeds into two operations. Its gradient must accumulate both contributions.

5.4 Derived Operations

Everything else is a composition of the basic rules:

- $a - b = a + (-b)$
- $a/b = a \cdot (b^{-1})$
- a^n — power rule
- $\tanh$, $\exp$, $\log$, ReLU — direct local rules

Each is just one more local derivative encoded in `_backward`.

5.5 Forward Pass Builds a Graph

Writing `loss = ((y_pred - y) ** 2).tanh()` constructs a tree of `Value` objects. Each has `_prev` pointing to its inputs. **The graph is the computation.**

5.6 Backward Pass Uses the Chain Rule

```

1 def backward(self):
2     topo, visited = [], set()
3     def build(v):
4         if v not in visited:
5             visited.add(v)
6             for c in v._prev: build(c)
7             topo.append(v)
8     build(self)
9     self.grad = 1.0
10    for v in reversed(topo):
11        v._backward()

```

Listing 5.4: The backward algorithm

1. **Topological sort**: order nodes so each comes after its dependencies.
2. **Seed**: set `self.grad = 1.0` because $\partial \text{loss} / \partial \text{loss} = 1$.
3. **Reverse walk**: call each node's `_backward()` to push gradient to parents.

Chapter 6

Backprop

6.1 The Setup

We have built a computation graph. `loss` is the root. Every parameter θ is a leaf. We want $\partial \text{loss} / \partial \theta$ for every leaf.



reverse order

The computation graph. Backprop walks from the output (root) to the parameters (leaves).

6.2 Topological Order

We need to process each node **after** every node that depends on it. Otherwise, when we call `node._backward()`, `node.grad` might not yet be complete. Topological sort guarantees this ordering. Visiting in reverse topological order means the root comes first, then its consumers, then down to the leaves.

6.3 Seeding and the Reverse Walk

Key Idea. `loss.grad = 1.0`. Because $\partial \text{loss} / \partial \text{loss} = 1$. Every other gradient is relative to this.

For each node, calling `_backward()`:

1. Reads `node.grad` (incoming gradient from all consumers—already accumulated).
2. Multiplies by each local derivative.
3. Adds results to each input's `.grad`.

This is the chain rule, evaluated in reverse, with accumulation at every node.

6.4 Why This Is Efficient

Naive: N forward passes for N parameters Backprop: 1 forward + 1 backward $\approx 2 \times$ forward
(6.1)

Key Idea. The chain rule is reused: intermediate results (activations, local derivatives) are computed **once** and shared across all paths that flow through them.

6.5 Backprop = Reverse-Mode Autodiff

Reverse mode: we walk the graph backward from the output.

Auto-diff: we don't write gradients by hand—the graph is differentiated automatically via each op's `_backward`.

6.6 The Full Picture

save inputs & value

value = op(inputs)

for each op in order:

output.grad = 1

for each op in reverse order:

All g

for each input: input.grad += local_deriv $\times$ output.grad

That is backprop. Everything else is engineering.

Chapter 7

Neural Networks

7.1 A Neuron

A neuron is a small differentiable function:

$$h = \tanh(\mathbf{w} \cdot \mathbf{x} + b) \quad (7.1)$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ is a weight vector, and b is a bias.

```
1 class Neuron:
2     def __init__(self, nin):
3         self.w = [Value(random.uniform(-1, 1) / (nin ** 0.5))
4                     for _ in range(nin)]
5         self.b = Value(0.0)
6
7     def __call__(self, x):
8         act = self.b
9         for xi, wi in zip(x, self.w):
10            act = act + xi * wi
11        return act.tanh()
12
13    def parameters(self):
14        return self.w + [self.b]
```

Listing 7.1: A single neuron

Note. Initialization. The $1/\sqrt{n_{\text{in}}}$ scaling keeps the dot product’s magnitude stable as n_{in} grows. Small detail, big practical difference.

7.2 Why Nonlinearity?

Without $\tanh$ (or ReLU), a stack of linear layers collapses into a single linear function:

$$W_2(W_1\mathbf{x} + b_1) + b_2 = (W_2W_1)\mathbf{x} + (W_2b_1 + b_2) = W\mathbf{x} + b \quad (7.2)$$

Caution. No matter how deep, it’s still linear. Nonlinearity is what lets deep networks represent nonlinear functions.

7.3 A Layer

A layer is n_{out} neurons, each seeing the same input:

```

1 class Layer:
2     def __init__(self, nin, nout):
3         self.neurons = [Neuron(nin) for _ in range(nout)]
4
5     def __call__(self, x):
6         outs = [n(x) for n in self.neurons]
7         return outs[0] if len(outs) == 1 else outs
8
9     def parameters(self):
10        return [p for n in self.neurons for p in n.parameters()]

```

Listing 7.2: A layer of neurons

7.4 The MLP

Stacking layers gives a multilayer perceptron:

```

1 class MLP:
2     def __init__(self, nin, nouts):
3         sizes = [nin] + nouts
4         self.layers = [Layer(sizes[i], sizes[i+1])
5                          for i in range(len(nouts))]
6
7     def __call__(self, x):
8         for layer in self.layers:
9             x = layer(x)
10        return x
11
12    def parameters(self):
13        return [p for layer in self.layers for p in layer.parameters()]

```

Listing 7.3: Multilayer perceptron

Note. Classification. For classification, we typically *do not* apply tanh on the last layer. We want raw logits for softmax cross-entropy.

7.5 Parameter Count

For MLP(64, [32, 32, 5]):

Table 7.1: Parameter count for MLP(64, [32, 32, 5]).

Layer	Formula	Parameters
Layer 1	$32 \times (64 + 1)$	2080
Layer 2	$32 \times (32 + 1)$	1056
Layer 3	$5 \times (32 + 1)$	165
Total		3301

This number is **fixed by architecture**, not by dataset size. Whether we feed 100 images or 100 million, the network has 3301 numbers to learn.

7.6 Every Forward Pass Is a Composition

Given input $\mathbf{x}$:

1. Compute $\mathbf{w}_1 \cdot \mathbf{x} + b_1$, apply $\tanh$.
2. Compute $\mathbf{w}_2 \cdot \mathbf{h}_1 + b_2$, apply $\tanh$.
3. Compute $\mathbf{w}_3 \cdot \mathbf{h}_2 + b_3$.
4. Return logits.

Every step is **Value** operations. The entire forward pass produces a graph of thousands of **Value** nodes. Backprop walks it in reverse **once**, and every parameter's gradient is filled in.

Chapter 8

The Loss

8.1 What a Loss Does

Key Idea. A loss function turns “how wrong is the model?” into a single number. It is the thing we minimize.

8.2 Cross-Entropy for Classification

For logits $\mathbf{z} = [z_1, \dots, z_C]$ and target class t :

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \text{loss} = -\log(\text{softmax}(\mathbf{z})_t) \quad (8.1)$$

The model predicts a probability distribution. Loss penalizes low probability on the correct class.

8.3 Why Log + Softmax?

Key Idea. The gradient with respect to the logits works out beautifully:

$$\frac{\partial \text{loss}}{\partial z_i} = \text{softmax}(\mathbf{z})_i - \mathbb{I}[i = t] \quad (8.2)$$

Predicted probability – one-hot target. Well-scaled, never vanishing, never exploding. Backprop derives this automatically—we never write it by hand.

8.4 Why Not MSE?

For classification, MSE with tanh-bounded outputs works but poorly:

- tanh saturates. When the model is confidently wrong, gradients vanish.
- MSE’s gradient on probabilities is small when the prediction is near 0 or 1, even if the answer is very wrong.

Cross-entropy avoids both. **The choice of loss shapes the optimization landscape—**even though backprop handles any differentiable loss equally.

8.5 Numerical Stability

e^z overflows for large z . Standard trick: subtract the max before exponentiating. Softmax is invariant to this shift.

```
1 def softmax_cross_entropy(logits, target_idx):
2     max_val = max(z.data for z in logits)
3     shifted = [z - max_val for z in logits] # stable
4     exps = [s.exp() for s in shifted]
5     total = exps[0]
6     for e in exps[1:]:
7         total = total + e
8     probs = [e / total for e in exps]
9     return -probs[target_idx].log()
```

Listing 8.1: Numerically stable softmax cross-entropy

Backprop handles this composition—exp, division, log—automatically.

Chapter 9

The Training Process

9.1 Step 1: Define the Model

```
1 model = MLP(64, [32, 32, 5])
2 params = model.parameters()
```

Listing 9.1: Model creation

9.2 Step 2: Prepare Data

Each sample is a pair (image, label), where the image is 64 pixels and the label is in {0, 1, 2, 3, 4}.

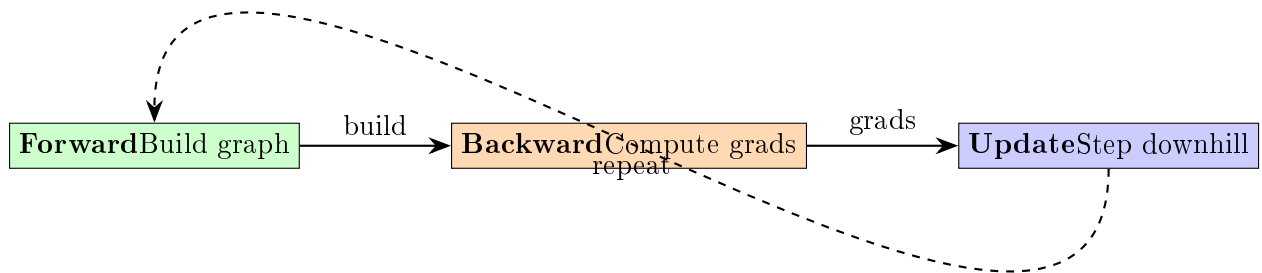
9.3 Step 3: The Loop

For each batch:

```
1 # --- Forward ---
2 batch_loss = Value(0.0)
3 for img, label in batch:
4     x = [Value(p) for p in img]
5     logits = model(x)
6     batch_loss = batch_loss + softmax_cross_entropy(logits, label)
7 batch_loss = batch_loss * (1.0 / len(batch))
8
9 # --- Backward ---
10 for p in params: p.grad = 0.0
11 batch_loss.backward()
12
13 # --- Update ---
14 for p in params:
15     p.data -= LEARNING_RATE * p.grad
```

Listing 9.2: Complete training step

9.4 The Three Phases



9.4.1 Zeroing Gradients

Because `_backward` uses `+=`, and `p.grad` may still hold yesterday's value. Without zeroing, gradients accumulate across epochs and training diverges.

9.5 What Changes as the Model Learns

Early epochs: loss decreases rapidly. Random weights become structured.

Middle epochs: the model finds useful features. Hidden-layer weights show edge and stroke detectors.

Late epochs: loss decreases slowly. The model fine-tunes.

If training and test loss start diverging, the model is **overfitting**—memorizing the training set rather than generalizing. The **generalization gap** (train loss $-$ test loss) is the central tension in machine learning.

Chapter 10

Putting It All to Work

10.1 The Dataset

Synthetic 8×8 images of digits 0–4, drawn as crude strokes with 10% noise. Flattened to 64 pixels.

Note. Why no convolutions? Our goal is to understand backprop, not build a state-of-the-art model. Convs add a new op with its own transposed-convolution backward, plus `im2col` and padding arithmetic—all of which obscure the core ideas. The backprop algorithm is **identical** with or without convolutions. Convs are just one more differentiable op.

10.2 Results

After ~ 40 epochs with $\eta = 0.05$:

Table 10.1: Training results for the digit classification task.

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
0	1.5861	0.281	1.5618	0.300
10	0.2210	0.881	0.3412	0.775
20	0.0781	0.963	0.2218	0.850
39	0.0281	0.991	0.1540	0.925

Train accuracy climbs to 99%. Test accuracy plateaus at 92.5%. The model learns.

10.3 What the Model Learned

Reshaping first-layer weights to 8×8 reveals stroke detectors—some neurons respond to horizontal bars, others to verticals, others to diagonal patterns. The second layer combines these into digit-part detectors. The third layer produces class scores.



This hierarchy emerges **automatically** from gradient descent on a differentiable function. We never told the model “look for strokes.” We told it “minimize the loss,” and backprop did the rest.

10.4 What We Skipped

Table 10.2: Features omitted for clarity, and their trade-offs.

Feature	Trade-off
Convolutions	Restore spatial structure, more op complexity
Vectorization	100 $\times$ faster, hides elementwise ops
Better optimizers	Same backprop, better gradient use
Regularization	Improve generalization, not optimization

All are **refinements on the same core loop**: forward, backward, step. The math is unchanged.

Summary

In One Paragraph

Every parameter in a neural network has a derivative: how the loss changes when you nudge that parameter.

Gradient descent uses these derivatives to step each parameter downhill.

The chain rule tells us how to compute the derivative of a composition—multiply local derivatives along the path.

Backprop is the chain rule applied systematically: build a graph on the forward pass, walk it backward to compute every parameter’s gradient in one pass.

A neural network is a composition of simple differentiable functions—linear combinations and nonlinearities.

Training is a loop: forward to compute loss, backward to compute gradients, update to nudge parameters.

Everything else—convs, attention, transformers—is a new differentiable op with a new local derivative.

The engine never changes.

Exercises

1. **Trace by hand.** Take a 2-input neuron with known weights and a known input. Compute forward and backward by hand. Verify with the code.
2. **Break +=.** Replace += with = in `__add__`'s `_backward`. Test on a graph where a value is used twice. Watch gradients come out wrong.
3. **Add ReLU.** Derive its local derivative. Implement `_backward`. Compare convergence to tanh.
4. **Remove gradient zeroing.** See training diverge. Understand why.
5. **Try MSE + tanh on classification.** Compare the loss curve. You'll see the well-known pathology: slow early training.
6. **Print the graph.** Write a function that walks `_prev` from a loss node and prints the operations. See the DAG.
7. **Swap in NumPy.** Wrap arrays instead of scalars. Same math, $100\times$ faster, new complexity: broadcasting.
8. **Add a conv.** Build a 3×3 filter that slides over the image. Derive its backward (a transposed convolution). Re-run the digit task. See the accuracy jump.

Each exercise targets a specific piece of intuition. Doing them all gives you the full picture.

References

Bibliography

- [1] Goodfellow I., Bengio Y., Courville A.
Deep Learning. MIT Press, 2016.
- [2] Rumelhart D. E., Hinton G. E., Williams R. J.
“Learning Representations by Back-propagating Errors.” *Nature*, 323:533–536, 1986.
- [3] LeCun Y., Bengio Y., Hinton G.
“Deep learning.” *Nature*, 521:436–444, 2015.
- [4] Mikolov T., Chen K., Corrado G., Dean J.
“Efficient Estimation of Word Representations in Vector Space.” *arXiv:1301.3781*, 2012.
- [5] Colt: *An Open-Source Auto-Differentiation Engine in C++*
(Inspiration for the tiny-grad style of autograd engine used in this text.)